

MODELO DE SÍLABO

MACROPROCESO DOCENCIA

Facultad de Ciencias Básicas y Tecnologías
Física

Sílabo
Programación

1. Descripción. La asignatura introduce la programación como herramienta de razonamiento y trabajo científico. Mediante Python, Linux, Git y bibliotecas del ecosistema científico, el estudiante aprende a traducir problemas físicos a procedimientos computacionales, leer y escribir código, procesar datos, producir visualizaciones, depurar, probar y validar resultados. El curso incorpora prácticas de reproducibilidad y formación explícita para evaluar críticamente la asistencia de inteligencia artificial.

El énfasis se encuentra en fundamentos transferibles. Los temas avanzados se seleccionan por su utilidad para la física y la ciencia de datos, evitando cubrir superficialmente aprendizaje automático, concurrencia u otras áreas que requieren cursos posteriores.

2. Justificación. La computación forma parte de la práctica contemporánea de la física junto con la teoría y el experimento. Un físico debe poder transformar modelos en código, trabajar con datos, comparar resultados con casos conocidos, cuantificar errores y comunicar evidencia. Estas competencias también permiten transitar hacia ciencia de datos, investigación interdisciplinaria y desarrollo de software científico.

El acceso generalizado a herramientas de IA aumenta la necesidad de que el estudiante construya competencia autónoma. Por ello, la asignatura combina evaluaciones individuales protegidas con actividades abiertas donde la IA puede ser estudiada, auditada y declarada sin reemplazar la responsabilidad humana.

3. Administración del espacio académico, núcleo o cátedra.

Generalidades	Detalle
Modalidad	Presencial
Espacio académico, núcleo o cátedra	Programación
Código	120710206
Tipo de Actividad Académica	Profesional
Naturaleza	Teórico-práctica
Enseñanzas	Unidad 1: Fundamentos de Programación en Python. Unidad 2: Fundamentos de Programación Orientada a Objetos (POO).

MODELO DE SÍLABO

MACROPROCESO DOCENCIA

	Unidad 3: Bibliotecas Científicas en Python. Unidad 4: Análisis y Ciencia de Datos.
Evaluación	Cuantitativa y cualitativa
Créditos	3 (tres)
Horas semanales	6 (seis)
Total de horas por semestre	196 (ciento noventa y dos)
Horas de docencia directa (HDD) o de trabajo interactivo (HTIC)	6 horas por semana
HDD/HTIC en docencia	6 horas por semana
HDD/HTIC en investigación	1 horas por semana
HDD/HTIC en extensión	0 horas por semana
Horas de trabajo independiente	6 horas por semana
Habitable	Sí
Validable	Sí
Homologable	Sí
Requisitos	Ecuaciones diferenciales

4. Relación de los alcances de la formación.

4.1. Unidades de competencia propias del espacio académico, núcleo o cátedra.

Unidad de competencia – UC2

El estudiante identifica e interpreta fenómenos de la física y resuelve problemas en los ámbitos teórico-experimentales, expresándolos de manera coherente y utilizando competencias comunicativas en lengua materna o alguna lengua extranjera.

4.2. Resultados de aprendizaje.

Se espera que al finalizar el curso el estudiante alcance los siguientes resultados:

- Explicar la relación funcional entre hardware, sistema operativo, sistema de archivos, procesos, intérprete de Python y entornos de ejecución.
- Utilizar una terminal Linux para organizar archivos, buscar información, combinar comandos y automatizar tareas sencillas con Bash.
- Emplear Git y GitHub para registrar cambios, recuperar versiones, colaborar y documentar un trabajo computacional.
- Descomponer un problema físico en entradas, supuestos, operaciones, algoritmos, resultados y criterios de validación.
- Leer, explicar y escribir programas en Python que utilicen tipos básicos, estructuras de control, colecciones, funciones, archivos, excepciones y módulos.

MODELO DE SÍLABO

MACROPROCESO DOCENCIA

- Depurar y verificar programas mediante mensajes de error, trazas, casos límite, pruebas automatizadas y comprobaciones físicas.
- Reconocer efectos de punto flotante, discretización, convergencia, unidades e incertidumbre en un resultado computacional.
- Elegir entre Python nativo, NumPy y Pandas de acuerdo con la estructura del problema y de los datos.
- Procesar arreglos y datos tabulares, y comunicar resultados mediante figuras científicas creadas con la interfaz orientada a objetos de Matplotlib.
- Formular, desarrollar, reproducir y defender un proyecto computacional con una pregunta física no trivial y evidencia de validación.
- Evaluar críticamente código y explicaciones producidos por IA, declarar su uso y conservar responsabilidad intelectual sobre el resultado.

5. Enseñanzas.

UNIDAD	CONTENIDO	DURACIÓN (semanas)
Unidad 1: Computador, entorno y reproducibilidad	• Componentes funcionales del computador. • Sistema operativo, procesos y sistema de archivos. • Linux y Bash: navegación, búsqueda, flujos, tuberías y scripts. • Git y GitHub: estados, commits, remotos y publicación básica. • Organización reproducible de un proyecto científico.	I - II
Unidad 2: Fundamentos de Programación	• Intérprete, scripts, notebooks y entornos virtuales. • Tipos, operadores, variables, entrada y salida. • Funciones, contratos, alcance, argumentos y documentación. • Booleanos, condicionales, ciclos y patrones de acumulación. • Cadenas y colecciones. • Archivos, rutas, excepciones, depuración, módulos y dependencias.	III - V

MODELO DE SÍLABO

MACROPROCESO DOCENCIA

	• Pruebas, casos límite, algoritmos elementales y error numérico.	
Unidad 3: Cómputo científico	• NumPy: arreglos, formas, tipos, indexación, vistas, broadcasting y vectorización. • Aleatoriedad reproducible. • Matplotlib mediante interfaz orientada a objetos. • Validación, convergencia, error e interpretación física. • Clases y objetos, herencia • SciPy, SymPy, SimPy y scikit-learn	VI - VII
Unidad 4: Datos e integración responsable	• Lectura de datos con Python nativo, NumPy y Pandas. • Selección, tipos, faltantes, duplicados, agrupación y combinación. • Incertidumbre, procedencia y comunicación científica. • Uso crítico de IA y agentes de programación. • Integración en un flujo científico pequeño.	VIII - XV
Unidad 5 — Integración y comunicación	• Demostración y sustentación de proyecto. • Discusión de límites, validación y reproducibilidad.	XVI

6. Procesos integrativos.

Los estudiantes tienen la opción de pertenecer a los semilleros de la Facultad y/o vincularse a los grupos de investigación del Programa mediante procesos de iniciación científica.

7. Metodología.

La asignatura combina explicaciones breves, programación en vivo, predicción de resultados, práctica guiada, trabajo en parejas, laboratorios y un proyecto aplicado. Una sesión típica alterna bloques de explicación de 15 a 25 minutos con actividades donde los estudiantes deben escribir, ejecutar, depurar o explicar código.

MODELO DE SÍLABO

MACROPROCESO DOCENCIA

Los notebooks se usan para exploración y comunicación; los componentes reutilizables se desarrollan en archivos Python y se prueban por separado. Git se usa durante todo el semestre, no únicamente como tema inicial.

Se programan seis laboratorios acumulativos. Cada uno exige solamente contenidos ya explicados y practicados. Además se planean ocho clases de recuperación sin fecha fija sobre NumPy, Matplotlib, Pandas, SciPy, SimPy, scikit-learn e integración. Son espacios de apoyo y no amplían el temario evaluable. La última semana, del 17 al 19 de noviembre, se dedica completa a sustentaciones y no introduce contenidos ni entregables nuevos.

El proyecto se desarrolla mediante propuesta, aprobación de alcance, demostración mínima, congelamiento de código y sustentación. Los proyectos pueden ser individuales o en parejas, pero la autenticación y parte de la nota son individuales.

8. Evaluación.

Se adopta el sistema descrito en [evaluacion.md](#):

- tres parciales escritos de 18 % cada uno;
- seis laboratorios de 3,5 % cada uno;
- proyecto final de 25 %, distribuido entre producto (12 %), sustentación y modificación explicada (8 %), y reproducibilidad, Git y declaración de IA (5 %).

Los parciales se realizan en papel. Las hojas de referencia suministradas por el docente reducen la dependencia de memorización de APIs y permiten evaluar trazado, diagnóstico, diseño, pruebas e interpretación. Cada parcial incluye como máximo lo visto el jueves de la semana anterior y excluye los temas del martes y miércoles de la semana del examen, como se detalla en [cronograma.md](#).

La rúbrica del proyecto se encuentra en [rubrica_proyecto_final.md](#).

9. Inteligencia artificial

El uso se rige por [politica_ia.md](#). Las evaluaciones protegidas no admiten IA ni autocompletado generativo. Las actividades autorizadas exigen declaración, revisión y evidencia independiente. Se enseña formalmente el trabajo con IA y agentes después de que el estudiante haya desarrollado competencias de prueba, depuración y análisis de datos.

10. Bibliografía.

Textos guía:

1. Nicolas Schurmann. *Ultimate Python: de cero a experto: Un libro para principiantes fácil de seguir*. Autoedición / Amazon Digital Services. 2024.
2. Nicolas Schurmann. *Ultimate Python: Avanzado*. Autoedición / Amazon Digital Services. 2024.
3. Nicolas Schurmann. *Ultimate Docker: guía de cero hasta despliegues*. Autoedición / Amazon Digital Services. 2024.

MODELO DE SÍLABO

MACROPROCESO DOCENCIA

4. Nicolas Schurmann. *Ultimate GIT + GITHUB: De la teoría a la práctica fácil de seguir*. Autoedición / Amazon Digital Services. 2024.
5. Brian Heinold. *Practical Introduction to Python Programming*. Department of Mathematics and Computer Science Mount St. Mary's University. 2012.
6. Jennifer Campbell, Paul Gries, Jason Montojo, Greg Wilson. *Practical Programming An Introduction to Computer Science Using Python 3*. Pragmatic Bookshelf. 2013.
7. John M. Zelle. *Python Programming: An Introduction to Computer Science*. Franklin, Beedle & Associates Inc. 2005.
8. Wes McKinney. *Python for data analysis*. O'Reilly. 2018.
9. Sandro Tosi. *Matplotlib for developers*. Packt Publishing. 2009.
10. Pablo M. García Corzo. *Proyecto Phythones Simulaciones Físicas en Visual Python*. Alqua. 2008.

Textos de lectura complementaria:

- 1- S. S. Skiena. *The Algorithm Design Manual*, Springer. 2008.
- 2- Z.A. Shaw. *Learn Python 3 the Hard Way*. Addison-Wesley Professional. 2013.
- 3- B. Miller, D. Ranum. *Problem Solving with Algorithms and Data Structures*, Franklin, Beedle & Associates, 2006.
- 4- A. Lancaster, G. Webster. *Python for the life sciences*. Apress. 2019.
- 5- J.M. Kinder, P. Nelson. *A student's guide to Python for physical modeling*. Princeton University Press. 2015.
- 6- Phuong Vo T H, Martin Czygan, Ashish Kumar, *Python: Data Analytics and Visualization: Understand, evaluate, and visualize data*, Packt Publishing. 2017
- 7- Zach Radtka, Donald Miner, *Hadoop with Python*, O'Reilly Media. 2015.
- 8- R. Wachenchauzer, M. Manterola, M. Curia, M. Medrano, N. Paez. *Algoritmos de Programación con Python*. Librosweb. Recuperado de <https://uniwebidad.com/libros/algoritmos-python> Recuperado el 17/02/2021
- 9- SQLModel, *SQL databases in Python, designed for simplicity, compatibility, and robustness* Recuperado de <https://sqlmodel.tiangolo.com/#license> el 14/02/2022
- 10- *FastAPI framework, high performance, easy to learn, fast to code, ready for production*, Recuperado de <https://fastapi.tiangolo.com/> el 14/02/2022
- 11- Ron White. *How Computers Work The Evolution of Technology*. QUE. 2014.
- 12- Google. *Google Python Style Guide*. Recuperado de: <https://google.github.io/styleguide/pyguide.html> recuperado en 22/07/2020.
- 13- Miktek. Recuperado de <https://miktex.org/download/ctan/systems/win32/miktex/setup/windows-x64/basic-miktex-2.9.7442-x64.exe> recuperado el 17/06/2020
- 14- Anaconda. Recuperado de <https://www.anaconda.com/products/individual#Downloads> recuperado el 17/06/2020

MODELO DE SÍLABO

MACROPROCESO DOCENCIA

11. Historial de revisión.

Primera versión: 2020-II

Revisión y actualización: 2021-I

Revisión y actualización: 2022-I

Revisión y actualización: 2024-II

Revisión y actualización: 2026-II

Vigencia del sílabo. Semestre II de 2026

Responsables:

Santiago Echeverri Arteaga, Ph.D.

Edgar Arturo Gómez, Ph.D.